

// JavaJub

Собес в Сбер

Java Middle

Вопросы, задачи и реальные кейсы
с технических интервью

Темы

JavaCore • JVM • Collections • Multithreading • Spring • SQL • System Design

!!"#\$%&'()*+&

Содержание

01	Как устроен отбор	,
02	Стек и темы, которые спрашивают	-
03	Java Core & ООП	.
04	Collections Framework	/
05	JVM, память, GC	0
06	Многопоточность и JMM	1
07	Spring и Spring Boot	22
08	Базы данных и SQL	23
09	Микросервисы и архитектура	2,
10	Практические задачи	2-
11	Поведенческое интервью	34
12	План подготовки + чек-лист	32

01 Как устроен отбор

В Сбере больше 2 000 продуктовых команд, и каждая может адаптировать процесс под себя. Но общая схема для Java Middle в 2025–2026 годах выглядит так:

Этап	Длительность	Что проверяют
HR-скрининг	30–40 мин	Мотивацию, ожидания по ЗП, общий опыт, стек
Тех-скрининг	30–45 мин	Базовый синтаксис, SQL, Big-O, коллекции
Java + Code review	60–90 мин	JavaCore, JVM, многопоточность, контракты
System Design	60–90 мин	Проектирование (для Middle+ — опционально)
Team Match	60 мин	Soft skills, ценности, выбор команды

► Как выглядит тех-секция

- Видео-созвон с демонстрацией экрана, совместный редактор кода.
- Обычно: теория + 1–2 практические задачи уровня LeetCode Easy/Medium или code review.
- Нужно рассуждать вслух — оценивается ход мыслей, а не только финальный ответ.
- AI-ассистенты и поисковики запрещены. За списыванием следят.

ФИШКА · Особенность Сбера

Реже дают чистые алгоритмы (в отличие от Яндекса и VK). Чаще — код с неочевидными багами, который компилируется и проходит тесты, но тихо ломает данные в проде. Смотрят, видишь ли ты контракты языка «под капотом».

02 Стек и темы, которые спрашивают

В 2026 году стандарт в Сбере — Java 21 LTS (местами уже Java 25), Spring Boot 3.x, PostgreSQL-совместимая Pangolin, Kafka-совместимая Corax, Kubernetes внутри Platform V.

► Обязательный минимум для Middle

- Java Core: ООП, equals/hashCode, Generics, Exceptions, Stream API, Optional
- Collections Framework: HashMap, ArrayList/LinkedList, ConcurrentHashMap, TreeMap
- JVM: heap/stack, сборщики мусора (G1, ZGC), JIT, classloading
- Многопоточность: synchronized, volatile, wait/notify, Executors, CompletableFuture, JMM
- Spring / Spring Boot: DI, bean scopes, @Transactional, REST, Spring Data JPA
- БД: SQL, индексы, уровни изоляции, MVCC, EXPLAIN, оптимизация запросов
- Микросервисы: REST, gRPC, Kafka, паттерны (Saga, Circuit Breaker, Idempotency)
- Тестирование: JUnit 5, Mockito, Testcontainers
- Git, Docker, основы Kubernetes, CI/CD

► Plusом будет

- Virtual Threads (Project Loom) — как влияют на производительность
- Records, Sealed classes, Pattern Matching (Java 17–21)
- Sequenced Collections (Java 21)
- Spring Native / GraalVM
- OpenTelemetry, observability
- Специфика Platform V: Pangolin, Synapse, Corax

03 Java Core & ООП

Самый частый блок. Для Middle ждут не только знания, но и понимания «почему так».

► Типовые вопросы

- Расскажите про ООП: инкапсуляция, наследование, полиморфизм.
→ Будьте готовы объяснить полиморфизм пятикласснику.
- В чём разница между композицией и агрегацией?
→ Композиция = владение (жизненные циклы связаны), агрегация = ссылка.
- Какие области памяти в JVM? Что хранится в heap, а что в stack?
→ Heap — объекты и массивы; stack — фреймы методов, примитивы, ссылки.
- Что такое String Pool? Почему String immutable?
→ Безопасность, потокобезопасность, кэширование hashCode, работа String Pool.
- Разница между == и equals(). Что вернёт someObj.equals(null)?
→ == сравнивает ссылки; equals(null) по контракту должен возвращать false.
- Контракт equals/hashCode. Что сломается при нарушении?
→ Если *a.equals(b)*, то hashCode совпадают. Иначе HashMap/HashSet теряют объекты.
- Что будет, если hashCode() всегда возвращает одну константу?
→ В Java 7 — деградация HashMap до связанного списка $O(n)$. В Java 8+ при 8 коллизиях в бакете он превращается в red-black tree, $O(\log n)$. В обоих случаях производительность резко падает.
- Разница между checked и unchecked исключениями.
→ Checked — от Exception (кроме RuntimeException), компилятор требует обработки. Unchecked — от RuntimeException/Error.
- try-with-resources — какой интерфейс нужен?
→ AutoCloseable (или Closeable). Ресурсы закрываются в обратном порядке.
- Что такое functional interface? Приведите 3 примера из java.util.function.
→ Интерфейс с одним абстрактным методом. Примеры: *Function<T,R>*, *Predicate<T>*, *Consumer<T>*, *Supplier<T>*.
- Stream API: промежуточные vs терминальные операции. Что такое ленивость?
→ Промежуточные (*filter*, *map*) не выполняются до вызова терминальной (*collect*, *forEach*).
- Разница между map и flatMap.
→ *map*: $T \rightarrow R$; *flatMap*: $T \rightarrow \text{Stream}<R>$ с «уплощением».
- Когда использовать Optional, а когда — нет?
→ Для возвращаемых значений. НЕ для полей, параметров методов, коллекций.
- Records (Java 16+): можно ли от них наследоваться?
→ Нет. Record неявно наследуется от *java.lang.Record*, а Java не поддерживает множественное наследование классов.
- Sealed classes — зачем?
→ Ограничение иерархии наследования. Хорошо работает с *pattern matching*.
- Generics: PECS. Что означает ? extends T и ? super T?
→ *Producer Extends*, *Consumer Super*. *extends* — можно читать, нельзя писать; *super* — наоборот.
- Что такое type erasure?

→ Стирание дженериков во время компиляции. Последствия: нельзя `new T()`, нельзя `instanceof T`, нельзя создать массив `T[]`.

СОВЕТ · Лайфхак

На любой вопрос про контракт `equals/hashCode` обязательно приведите пример: «если я положу объект в `HashSet`, а потом изменю поле, участвующее в `hashCode`, — я его больше не найду». Это показывает понимание, а не заученность.

04 Collections Framework

HashMap — абсолютный чемпион по числу упоминаний. Спрашивают все и всегда.

- Как устроен HashMap? Что такое bucket, как разрешаются коллизии?
→ Массив связанных списков (*Node[]*). Бакет = ячейка массива. Коллизия → цепочка.
- Что поменялось в HashMap с Java 8?
→ При 8 элементах в бакете и размере таблицы ≥ 64 список превращается в *red-black tree*. Поиск становится $O(\log n)$ вместо $O(n)$ в худшем случае.
- Сложность операций HashMap: put, get, remove.
→ Амортизированная $O(1)$, в худшем случае $O(\log n)$ (Java 8+) или $O(n)$ (Java 7).
- Что такое load factor?
→ Порог заполнения (по умолчанию 0.75), при котором происходит *resize* — удвоение массива и *rehashing* всех элементов.
- Можно ли использовать как ключ изменяемый объект (например, массив)?
→ Технически можно, но опасно: если изменить объект после добавления — его *hashCode* может измениться, и найти элемент станет невозможно.
- HashMap vs ConcurrentHashMap — ключевые различия.
→ HashMap не потокобезопасен, допускает null-ключ/значение. ConcurrentHashMap потокобезопасен, не допускает null, в Java 7 использовал сегментную блокировку, в Java 8+ — CAS + *synchronized* на головах бакетов.
- HashMap vs TreeMap vs LinkedHashMap — когда что?
→ HashMap — быстрый доступ без порядка. TreeMap — сортировка ключей, $O(\log n)$. LinkedHashMap — сохраняет порядок вставки или *access order*.
- ArrayList vs LinkedList — где быстрее вставка в середину?
→ У LinkedList $O(1)$ на саму вставку, но $O(n)$ на поиск позиции. В реальности ArrayList почти всегда быстрее из-за локальности в кэше CPU.
- Как работает resize в ArrayList?
→ Новый массив в 1.5 раза больше, копирование через *Arrays.copyOf*.
- Fail-fast vs fail-safe итераторы.
→ Fail-fast (HashMap, ArrayList) кидают *ConcurrentModificationException* при изменении коллекции. Fail-safe (ConcurrentHashMap, CopyOnWriteArrayList) работают со снимком.
- List.of(1,2,3) vs new ArrayList<>() — в чём разница?
→ List.of возвращает неизменяемый список. add/remove бросают *UnsupportedOperationException*.
- Sequenced Collections (Java 21) — что это?
→ Новый интерфейс с методами *getFirst*, *getLast*, *addFirst*, *addLast*, *reversed*. Унифицирует работу с упорядоченными коллекциями.

05 JVM, память, GC

- JVM, JRE, JDK — в чём разница?
→ JVM — виртуальная машина. JRE = JVM + библиотеки. JDK = JRE + инструменты разработки (javac, jdb).
- Области памяти JVM.
→ Heap (shared), Stack (per thread), Metaspace (class metadata), PC Register, Native Method Stack.
- Young / Old generation — как работает поколенческая сборка?
→ Young: Eden + 2 Survivor. После нескольких переживаний minor GC — объект попадает в Old.
- Какие бывают сборщики мусора?
→ Serial, Parallel, CMS (deprecated с Java 9, удалён в Java 14), G1 (default с Java 9), ZGC, Shenandoah. Для low-latency выбирают ZGC/Shenandoah.
- Что такое Stop-the-World?
→ Пауза приложения для сборки мусора. У ZGC и Shenandoah паузы < 10 мс даже на огромных heap'ах.
- Виды OutOfMemoryError.
→ Java heap space, Metaspace, Unable to create new native thread, GC overhead limit exceeded.
- Как диагностировать утечку памяти?
→ Heap dump (jmap, -XX:+HeapDumpOnOutOfMemoryError) → анализ в Eclipse MAT или VisualVM.
- Classloader: parent delegation model.
→ Bootstrap → Platform (ext) → Application. Дочерний сначала делегирует родителю, потом грузит сам.
- Что такое JIT-компиляция?
→ Оптимизация «горячего» кода в нативный. C1 — быстрый прогрев, C2 — агрессивная оптимизация.
- Escape analysis — что это даёт?
→ JIT определяет, «сбегает» ли объект из метода. Если нет — может аллоцировать на стеке вместо heap (scalar replacement).

06 Многопоточность и JMM

Для Middle этот блок — обязательный. Готовьтесь подробно.

- Чем процесс отличается от потока?
→ Процесс — изолированная единица ОС со своей памятью. Поток — единица исполнения внутри процесса, память общая.
- Способы создать поток в Java.
→ *extends Thread, implements Runnable / Callable, ExecutorService, CompletableFuture, Virtual Thread (Java 21+).*
- Жизненный цикл потока.
→ *NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED.*
- Что такое монитор объекта? Как работает synchronized?
→ *У каждого объекта есть монитор. synchronized захватывает его при входе, освобождает при выходе (включая исключения).*
- Разница между synchronized-методом и synchronized-блоком.
→ *Метод захватывает this (для static — Class-объект). Блок — любой указанный объект. Блок обычно эффективнее.*
- wait / notify / notifyAll — правила использования.
→ *Вызывать только внутри synchronized на том же объекте. wait() освобождает монитор и засыпает. notify будит один поток, notifyAll — все.*
- Почему wait() нужно проверять в while, а не if?
→ *Из-за spurious wakeup — поток может проснуться сам по себе. После пробуждения условие может быть уже невалидным.*
- Зачем нужен volatile?
→ *Гарантирует видимость изменений между потоками и запрет reorderings. НЕ гарантирует атомарность составных операций (i++).*
- Что такое happens-before?
→ *Отношение в JMM, гарантирующее видимость операций. Примеры: synchronized release→acquire, volatile write→read, Thread.start→действия потока.*
- Atomic-классы — как работают без блокировок?
→ *Через CAS (compare-and-swap) — атомарную инструкцию процессора. Lock-free алгоритм.*
- Что такое ABA-проблема?
→ *Значение сменилось с A на B и обратно на A. CAS это не видит. Решение: AtomicStampedReference с версиями.*
- Виды пулов потоков в Executors.
→ *newFixedThreadPool, newCachedThreadPool, newSingleThreadExecutor, newScheduledThreadPool, newWorkStealingPool. Для прода обычно создают ThreadPoolExecutor вручную, чтобы контролировать очередь.*
- Почему опасно использовать Executors.newCachedThreadPool() в проде?
→ *Unbounded очередь и неограниченное количество потоков — при всплеске нагрузки можно положить JVM OutOfMemoryError.*
- ThreadLocal — где пригождается и где опасен?
→ *Удобен для контекста (например, SecurityContext). Опасен при использовании с пулами потоков — значение «протекает» между задачами. Обязательно remove() в finally.*
- CompletableFuture: thenApply vs thenCompose vs thenCombine.

→ *thenApply*: преобразование результата. *thenCompose*: *flatMap* для *CompletableFuture*.
thenCombine: объединение двух результатов.

- Virtual Threads — чем отличаются от обычных?

→ *Управляются JVM, а не ОС. Лёгкие (несколько KB против ~1 MB у platform). Можно писать блокирующий код, который масштабируется как асинхронный.*

- Как сделать потокобезопасный singleton?

→ *Лучшее — enum. Альтернативы: static holder (lazy init via classloader), double-checked locking с volatile.*

- Deadlock, livelock, starvation — в чём разница?

→ *Deadlock: два потока ждут друг друга. Livelock: потоки активны, но не прогрессируют. Starvation: поток не получает ресурс.*

ЛОВУШКА · Классика Сбера

Попросить написать double-checked locking singleton. Если кандидат забыл volatile на instance — это минус. Спросите себя: зачем здесь volatile? (Ответ: без него возможна публикация «полусозданного» объекта другим потоком из-за reordering).

07 Spring и Spring Boot

- Что такое IoC и DI?
→ IoC — принцип: контейнер управляет объектами. DI — реализация: зависимости внедряются извне (конструктор/сеттер/поле).
- Какой способ внедрения предпочтительнее и почему?
→ *Constructor injection: immutable-поля (final), явные обязательные зависимости, удобство в тестах без mock-контейнера, нет циклических зависимостей.*
- Жизненный цикл бина.
→ *Создание → @PostConstruct → InitializingBean.afterPropertiesSet → custom init → использование → @PreDestroy → DisposableBean.destroy.*
- Scopes бина. Какой default?
→ *singleton (default), prototype, request, session, application, websocket.*
- Как работает @Transactional под капотом?
→ *Spring создаёт прокси (JDK dynamic proxy или CGLIB). При вызове метода через прокси — открывается транзакция, в finally — commit/rollback.*
- Почему @Transactional не работает при вызове метода того же класса?
→ *Self-invocation идёт минуя прокси. Решение: вынести метод в отдельный бин или inject self.*
- Propagation-уровни транзакций.
→ *REQUIRED (default) — присоединяется или создаёт. REQUIRES_NEW — всегда новая. NESTED — savepoint. SUPPORTS, MANDATORY, NOT_SUPPORTED, NEVER.*
- Уровни изоляции транзакций.
→ *READ_UNCOMMITTED, READ_COMMITTED (default в PostgreSQL), REPEATABLE_READ, SERIALIZABLE. Защита от dirty/non-repeatable/phantom read.*
- Как работает @ComponentScan?
→ *Сканирует пакеты (по умолчанию — пакет класса с @SpringBootApplication) в поисках @Component, @Service, @Repository, @Controller.*
- Spring Boot auto-configuration — как устроен?
→ *@EnableAutoConfiguration + @Conditional. Конфиги описаны в META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports (ранее — spring.factories).*
- N+1 проблема в Spring Data JPA.
→ *При fetch = LAZY на связи, при итерации по списку делается +1 запрос за каждую связанную сущность. Решения: JOIN FETCH, @EntityGraph, BatchSize.*
- Чем @Async может удивить?
→ *Не работает self-invocation (как у @Transactional). Нужен @EnableAsync. Default executor может быть неподходящим — часто настраивают свой.*

08 Базы данных и SQL

В Сбере — PostgreSQL-совместимая Pangolin. Ждут реальное знание Postgres, а не «умею писать SELECT *».

- ACID — расшифровать.
→ *Atomicity (всё или ничего), Consistency (целостность), Isolation (параллельные транзакции), Durability (сохранность после commit).*
- Уровни изоляции и какие проблемы решают.
→ *READ_UNCOMMITTED — dirty read. READ_COMMITTED — non-repeatable read. REPEATABLE_READ — phantom read. SERIALIZABLE — полная сериализация.*
- Dirty / non-repeatable / phantom read — пример каждого.
→ *Dirty: чтение незакоммиченных данных. Non-repeatable: два чтения одной строки дают разный результат. Phantom: два чтения с WHERE возвращают разное число строк.*
- Типы индексов в PostgreSQL.
→ *B-tree (default, для =, <, >, BETWEEN), Hash (только =), GIN (массивы, JSONB), GiST (геометрия), BRIN (большие таблицы с порядком), SP-GiST.*
- Covering index — что это?
→ *Индекс, который содержит все поля запроса (через INCLUDE в PostgreSQL 11+). Позволяет Index Only Scan — без обращения к таблице.*
- Как прочитать EXPLAIN ANALYZE?
→ *Обратите внимание на Seq Scan (плохо на больших таблицах), Index Scan, Bitmap Heap Scan, Nested Loop vs Hash Join, Rows Removed by Filter, разницу estimated vs actual rows.*
- MVCC в PostgreSQL — как работает?
→ *Каждая транзакция видит snapshot базы. Вместо изменений — новые версии строк с xmin/xmax. Старые версии чистит VACUUM.*
- Pessimistic vs optimistic locking.
→ *Pessimistic: SELECT ... FOR UPDATE блокирует строку. Optimistic: версия (@Version) — при commit проверяется, изменилась ли она.*
- N+1 в JPA — как диагностировать?
→ *Включить логи SQL (spring.jpa.show-sql=true), смотреть количество запросов на эндпоинт, использовать pbspy или Hibernate Statistics.*
- Шардирование vs репликация.
→ *Репликация — копия данных для отказоустойчивости и масштабирования чтения. Шардирование — разделение данных по узлам для масштабирования записи.*

09 Микросервисы и архитектура

- Монолит vs микросервисы — когда что?
→ Монолит проще разрабатывать и деплоить, микросервисы — масштабировать независимо и развивать большими командами. Начинайте с монолита.
- SOLID — разобрать каждый принцип.
→ SRP (одна причина для изменения), OCP (расширение без модификации), LSP (подтипы взаимозаменяемы), ISP (мелкие интерфейсы), DIP (зависимость от абстракций).
- CAP-теорема. В финтехе что важнее?
→ При партиционировании выбираем между Consistency и Availability. В платежах обычно $C > A$ (лучше отказать, чем списать дважды).
- Распределённые транзакции: 2PC vs Saga.
→ 2PC — координатор + участники, блокировки, плохо масштабируется. Saga — цепочка локальных транзакций с компенсирующими действиями. Хореография vs оркестрация.
- Idempotency Key — зачем в платежах?
→ Клиент шлёт повтор — сервер распознаёт по ключу и не выполняет операцию дважды.
- REST vs gRPC — когда что?
→ REST — публичные API, простая отладка. gRPC — внутренний inter-service (быстрее, типизация через protobuf, streaming).
- Kafka: partition, offset, consumer group.
→ Partition — шард топика. Offset — позиция в partition. Consumer group — каждый partition читаем один consumer из группы.
- At-least-once vs at-most-once vs exactly-once.
→ В реальности чаще at-least-once + идемпотентность на стороне consumer'а. Kafka с transactional producer + read_committed даёт effectively-once.
- Circuit Breaker — зачем?
→ Защита от каскадных падений. При N ошибках подряд «размыкается» — запросы сразу отклоняются, сервис получает время оправиться.

10 Практические задачи

Ключевая особенность — любят код с неочевидными багами и просят code review.

► Задача 1. Code Review класса Person (★ легенда Сбера)

Формулировка: перед вами класс Person с аннотациями Serializable, Externalizable, методом clone() и JAXB-разметкой. В коде 8 багов. Найдите все, объясните последствия, предложите фикс. Время — 20 минут.

Чек-лист багов, которые нужно увидеть

1. Нет no-arg конструктора → при десериализации через Externalizable упадёт `InvalidClassException`. Externalizable требует публичный конструктор без параметров.
2. В `writeExternal/readExternal` поля пишутся и читаются в разном порядке (например: `write name→surname→phone→address`, `read name→surname→address→phone`). Компилируется, тесты на null-полях зелёные, а в проде у клиента в поле «адрес» записывается телефон.
3. `clone()` объявлен, но класс не реализует `Cloneable` → `CloneNotSupportedException`, обёрнутый в `Error`.
4. Поверхностное клонирование (shallow copy) — коллекции и Date-поля делятся между оригиналом и копией.
5. `hashCode()` возвращает константу (например, 1). Все объекты падают в один bucket. В Java 8+ — $O(\log n)$ через red-black tree, но всё равно деградация производительности.
6. `equals()` нарушает контракт: использует не те поля, что `hashCode`, или не симметричен/не транзитивен.
7. Mutable поле (`java.util.Date`) возвращается наружу без defensive copy — клиент может его изменить.
8. JAXB-аннотации на final-поле без сеттера / не тот access type → маршаллинг работает некорректно или вообще падает.

ЛОВУШКА • Самый коварный баг

Перепутанный порядок полей в `writeExternal/readExternal`. Всё компилируется, все тесты проходят — но в проде данные тихо переезжают из поля в поле. В банковском софте это стоит денег.

► Задача 2. Окно транзакций

Формулировка: дан отсортированный по возрастанию массив timestamp'ов транзакций (в секундах) и длина окна N секунд. Найдите максимальное количество транзакций, попадающих в любое окно длиной $\leq N$.

Эталон: техника «два указателя»

```
public int maxTransactions(int[] timestamps, int n) {
    int left = 0;
    int maxCount = 0;
    for (int right = 0; right < timestamps.length; right++) {
        while (timestamps[right] - timestamps[left] > n) {
            left++;
        }
        maxCount = Math.max(maxCount, right - left + 1);
    }
    return maxCount;
}
```

Сложность: $O(n)$ по времени, $O(1)$ по памяти. Оба указателя в сумме делают не больше $2n$ шагов.

О чём обязательно сказать

- Edge cases: пустой массив, один элемент, все timestamp'ы одинаковые.
- Допущение: массив отсортирован. Если нет — добавить `Arrays.sort`, $O(n \log n)$.
- Переполнение при большом N: использовать long, если timestamps — это миллисекунды.

► Задача 3. Rate Limiter

Формулировка: реализуйте компонент, ограничивающий запросы: не более K запросов от одного пользователя за окно T секунд. Сервис многопоточный. Обсудите хотя бы два алгоритма.

Варианты, которые нужно обсудить

- Fixed Window Counter — простой, но «скачок» на границе окна (можно получить 2K запросов за короткий промежуток).
- Sliding Window Log — точный, но хранит timestamp каждого запроса (дорогая память).
- Sliding Window Counter — гибрид: процент текущего окна + процент предыдущего.
- Token Bucket — выдача токенов с фиксированной скоростью, запрос тратит токен.

Скелет in-memory реализации

```
public class SlidingWindowRateLimiter {
    private final int limit;
    private final long windowMs;
    private final ConcurrentHashMap<String, Deque<Long>> userRequests =
        new ConcurrentHashMap<>();

    public SlidingWindowRateLimiter(int limit, long windowMs) {
        this.limit = limit;
        this.windowMs = windowMs;
    }

    public boolean allow(String userId) {
        long now = System.currentTimeMillis();
        Deque<Long> window = userRequests.computeIfAbsent(
            userId, k -> new ArrayDeque<>());
        synchronized (window) {
```

```
        while (!window.isEmpty() && window.peekFirst() <= now - windowMs) {  
            window.pollFirst();  
        }  
        if (window.size() >= limit) return false;  
        window.addLast(now);  
        return true;  
    }  
}
```

Подвохи, которые спросят

- Утечка памяти: Deque для неактивных пользователей. Нужен фоновый cleanup.
- Распределённость: в кластере нужен Redis с атомарными INCR + EXPIRE или Lua-скриптом.
- synchronized на Deque — правильно ли, если сам map уже concurrent? Да: map защищает только структуру, но не внутреннее состояние Deque.

► Задача 4. Упрощённая ConcurrentHashMap с сегментами

Формулировка: напишите потокобезопасный Map с сегментной блокировкой. Не JDK-версию один-в-один, а упрощённую — важно показать принцип.

ФИШКА · Контекст вопроса

Сегменты (класс Segment) — это реализация Java 7. В Java 8+ ConcurrentHashMap использует CAS + synchronized на головах бакетов. Задача учебная — показать понимание, зачем вообще нужна шардированная блокировка.

Идея решения

- Массив сегментов (например, 16 штук). Каждый сегмент — свой HashMap с собственным lock.
- Сегмент выбирается по хэшу ключа: `segments[hash(key) & (segmentCount - 1)]`.
- Потоки, работающие с разными сегментами, не мешают друг другу.

Типовые ошибки

- Начать с «обернём HashMap в synchronized» — это совсем не то, что спрашивают.
- Не объяснить, почему Java 8 ушла от этой схемы (сегменты — это дополнительный уровень косвенности и память).

► Задача 5. Свой HashMap

Формулировка: реализуйте MyHashMap<K, V> с методами put, get, remove. Опишите, как разрешаются коллизии и когда делать resize.

Что должно быть в решении

- Массив Node[] — bucket'ы.
- Node хранит key, value, hash, next (связный список для коллизий).
- Выбор бакета: $(n - 1) \& \text{hash}$, где n — степень двойки.
- Load factor 0.75 — при $\text{size} \geq \text{capacity} * 0.75$ делаем resize (удваиваем массив и rehash).
- Обработка null-ключа (в JDK HashMap допускается один null-ключ в bucket 0).
- Правильное использование equals и hashCode ключа.

► Задача 6. Producer-Consumer без BlockingQueue

Формулировка: классическая задача на wait/notify. BlockingQueue использовать нельзя.

```
public class BoundedBuffer<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public BoundedBuffer(int capacity) {
        this.capacity = capacity;
    }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) {
            wait();
        }
        queue.add(item);
    }
}
```

```
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) {
            wait();
        }
        T item = queue.poll();
        notifyAll();
        return item;
    }
}
```

Ключевые моменты, на которые смотрит интервьюер

- while, а не if — защита от spurious wakeup.
- notifyAll, а не notify — иначе при нескольких producer'ах и consumer'ах возможны потерянные сигналы и зависание.
- wait обязательно внутри synchronized, иначе IllegalMonitorStateException.
- InterruptedException пробрасывается — поток имеет право быть прерванным.

► Задача 7. SQL про транзакции

Формулировка: есть таблицы `accounts(id, user_id, balance)` и `transactions(id, from_account, to_account, amount, created_at)`. Найдите пользователей, у которых за последние 24 часа суммарно ушло со счетов больше 1 млн рублей. Отсортируйте по сумме по убыванию.

Эталон (PostgreSQL)

```
SELECT a.user_id, SUM(t.amount) AS total_out
FROM transactions t
JOIN accounts a ON a.id = t.from_account
WHERE t.created_at >= NOW() - INTERVAL '24 hours'
GROUP BY a.user_id
HAVING SUM(t.amount) > 1000000
ORDER BY total_out DESC;
```

Почти наверняка спросят

- Какие индексы поставить? — составной на `(from_account, created_at)` или `(created_at) + (from_account)`.
- Почему фильтр `created_at` стоит в `WHERE`, а не в `HAVING`? — `WHERE` режет строки ДО агрегации, `HAVING` — ПОСЛЕ. В `WHERE` работают индексы, в `HAVING` — нет.
- Как переписется на оконные функции, если нужно не суммы по пользователю, а ТОП-N крупнейших транзакций в каждом часе?

► Задача 8. System Design (для Middle+)

Популярные формулировки из реальных собеседований:

- Спроектируйте систему мгновенных переводов между банками (аналог СБП).
- Спроектируйте систему кэшбэка для миллионов пользователей.
- Спроектируйте URL-shortener под нагрузку 10 000 RPS.
- Спроектируйте систему уведомлений: email/SMS/push с фильтрацией каналов и дедупликацией за 24 часа.

Обязательный каркас ответа

9. Уточнить требования: функциональные + нагрузка (RPS, объём, latency).
10. Back-of-the-envelope: RPS → IOPS → storage per day/year.
11. High-level components: клиент → LB → API Gateway → сервисы → хранилища.
12. Схема API: REST или gRPC, ключевые эндпоинты.
13. Выбор хранилищ с обоснованием (Postgres для транзакций, Redis для кэша, Kafka для событий).
14. Consistency vs availability: для финтеха чаще выбираем consistency.
15. Отказоустойчивость: репликация, шардирование, Circuit Breaker, retry с jitter.
16. Безопасность: шифрование at-rest и in-transit, RBAC, аудит, маскирование ПДн в логах.
17. Масштабирование: узкие места и как их расширять.
18. Наблюдаемость: какие метрики, логи, трейсы.

11 Поведенческое интервью

В Сбере hard skills — это примерно 70% успеха. Остальное — культурный фит. Отмечается: при прочих равных кандидат с чуть более слабой техникой, но с хорошей коммуникацией, получает оффер чаще.

► Вопросы, которые реально задают

- Какая самая интересная задача была на прошлой работе? Расскажите от постановки до результата.
- Расскажите про баг в проде, который вы нашли и исправили.
- Опишите ситуацию конфликта в команде и как вы её разрешили.
- Почему уходите с текущего места? Почему именно Сбер?
- Что вы делаете, когда аналитик приносит противоречивые требования?
- Как оцениваете срок задачи? Что делаете, если не успеваете?
- Опишите проект, которым гордитесь. Ваша роль в нём?
- С каким типом коллег работать тяжелее всего? Как справляетесь?

► Формат ответа: STAR

- Situation — контекст: проект, команда, обстоятельства.
- Task — какая перед вами стояла задача.
- Action — что конкретно сделали Вы (не «мы»).
- Result — результат с цифрами: «ускорил на 40%», «снизил latency с 800 до 120 мс».

СОВЕТ • Сильный ответ

«В сервисе платежей при всплеске нагрузки появлялись таймауты. Я снял thread dump, увидел блокировку на synchronized в кэше курсов валют. Заменяю на ConcurrentHashMap с computeIfAbsent — p99 упал с 1200 до 180 мс, инцидент на бизнес-метрику исчез».

Конкретика + цифры + ваша роль.

12 План подготовки + финальный чек-лист

► За 3–4 недели

- 19. Перечитать Effective Java (Joshua Bloch), особенно главы про equals/hashCode, сериализацию и immutable-объекты — в Сбере это спрашивают постоянно.
- 20. Прорешать 20–30 задач на LeetCode (Easy + Medium): массивы, хэш-таблицы, два указателя, скользящее окно, деревья.
- 21. Повторить JVM и GC по докладом Алексея Шипилёва на YouTube.
- 22. Поднять локально проект на Spring Boot 3.x с PostgreSQL и Testcontainers, чтобы свежо помнить практику.

► За неделю

- 23. 2–3 мок-интервью (друзья, pramp.com, interviewing.io).
- 24. 5–10 разборов System Design (YouTube: «URL shortener», «payment system»), адаптируйте под финтех.
- 25. Подготовьте 3–4 истории по STAR из реального опыта.
- 26. Изучите основы Platform V, Pangolin, Corax — даже поверхностное знание выделит вас.

► В день собеседования

- Проверьте камеру, микрофон, стабильность интернета заранее.
- Держите лист бумаги — иногда проще нарисовать схему.
- Рассуждайте вслух: интервьюеру важен ход мысли.
- Не бойтесь задавать уточняющие вопросы по условию.
- В тупике — скажите об этом, попросите подсказку. Это не провал.
- В конце — 2–3 вопроса о команде, проекте и процессах.

ВНИМАНИЕ · Предупреждение

AI-ассистенты и поисковики во время собеседования запрещены. По словам интервьюеров Сбера, это заметно — слишком ровные ответы, задержки перед каждой фразой, взгляд в сторону экрана. Репутационные последствия серьёзные.

► Финальный чек-лист

Блок	Готов, если можешь...
Java Core	Объяснить контракт equals/hashCode на примере и показать, как его нарушение ломает HashSet
Collections	Рассказать, как устроен HashMap в Java 8+, и что делать, чтобы не получить O(n)
JVM	Назвать области памяти, виды OOM и как их диагностировать
Многопоточность	Написать потокобезопасный singleton 3 способами и объяснить роль volatile

JMM	Объяснить happens-before на примере volatile и synchronized
Spring	Рассказать, почему @Transactional не работает при self-invocation
SQL	Прочитать EXPLAIN, предложить индекс и объяснить, почему WHERE лучше HAVING
Архитектура	Объяснить CAP-теорему и выбор для платежей
Практика	Написать sliding window и rate limiter с thread safety
Soft	Рассказать 3 истории по STAR с конкретными цифрами

Удачи на собеседе!

// git push origin offer --force

JavaJub